

MASTER IMPLEMENTATION PLAN

FPGA NxN CNN Convolution Accelerator

IEEE SSCS Egypt 2026 — 12 Independently-Verifiable Milestones,
Software Through Hardware

Target platform: ZedBoard (Zynq-7000, xc7z020)

Toolchain: Xilinx Vivado 2025.2 · PetaLinux

RTL: VHDL | **Software:** Python (PyTorch)

Weight source: grayscale CIFAR-10, trained first layer (16–64 filters), 4 selected + quantized

How to read this plan: every milestone lists the exact files/scripts it produces and a standalone verification gate. No milestone requires the ones after it to be tested — only the ones before it.

Table of Contents

- 0. Repository, Environment & Directory Skeleton — Setup
- 1. CIFAR-10 Grayscale Training Pipeline — Software
- 2. Filter Selection & Quantization / Export Pipeline — Software
- 3. Golden Reference Model & Test-Vector Generation — Software
- 4. RTL — Configuration Register Bank & AXI4-Lite Control — Hardware
- 5. RTL — Shared Window Generator — Hardware
- 6. RTL — Per-Channel Compute Datapath — Hardware
- 7. RTL — Top-Level K-Channel Integration — Hardware
- 8. Vivado Block Design, AXI DMA, Synthesis & Implementation — System
- 9. PetaLinux BSP & Device Tree — System
- 10. PS-Side Python Driver & On-Target Hardware Verification — Software
- 11. Full System Demo, Bonus Features & Report Packaging — Final

Parallelization note: M1–M3 (training/export/golden-model, software) and M4–M6 (RTL submodules, hardware) have no dependency on each other and can be built simultaneously — they only need to converge once, at M7. M2's real trained weights aren't required until M10; M6/M7's RTL verification can run against generic/hardcoded-kernel vectors from M3 in the meantime.

0. Repository, Environment & Directory Skeleton

SETUP

Objective: Establish a single, consistent project layout up front so every later milestone has an unambiguous, agreed-upon home for its files — no reorganizing later.

File	Type	Purpose
README.md	Docs	Direct instructions for contributors, build instructions, and links to this repository
requirements.txt	Python	torch, torchvision, numpy, matplotlib — pinned versions
training/, export/, golden_model/, rtl/, tb/, vivado/, petalinux/, ps_app/, demo/, report/	Dirs	One directory per milestone to track files in. Append A)

STANDALONE VERIFICATION

No functional test — the check here is a checklist: repo initialized, skeleton committed, README describes what goes where. Every later milestone's "Files" column assumes this layout exists.

1. CIFAR-10 Grayscale Training Pipeline

SOFTWARE

Depends on: M0

Objective: Produce a real, reasonably-accurate trained CNN whose first convolutional layer will later be extracted and deployed — grayscale input, standard-width first layer (16–64 filters), as decided.

File	Type	Purpose
training/dataset.py	Python	Loads CIFAR-10, applies grayscale conversion (<code>torchvision Grayscale()</code>), builds train/val loaders
training/model.py	Python	CNN definition: first conv = 3×3, padding='same', bias=True, 16–64 out_channels, followed by enough additional layers to train a legitimate classifier
training/train.py	Python	Training loop, checkpointing, logging
training/evaluate.py	Python	Validation-accuracy check against held-out CIFAR-10 test set

STANDALONE VERIFICATION

Train to completion and confirm validation accuracy lands in a reasonable range for a small grayscale CIFAR-10 CNN (a sanity bar, not a competition metric — e.g. comfortably above chance and in the neighborhood of published small-CNN grayscale baselines). Confirm a saved checkpoint reloads and reproduces the same validation accuracy. **Gate: this milestone is entirely software, needs no hardware or golden model, and is fully testable in isolation.**

2. Filter Selection & Quantization / Export Pipeline

SOFTWARE

Depends on: M1

Objective: Turn the trained model's first layer into exactly what the hardware needs: 4 selected filters, each quantized to an 8-bit signed kernel + bias + shift, in the exact numeric convention the RTL will implement.

File	Type	Purpose
export/select_filters.py	Python	Implements the diversity-based selection method: picks 4 maximally-distinct filters from the trained (wider) first layer
export/quantize.py	Python	Per-channel max-abs calibration — finds each channel's largest shift S s.t. $\text{round}(\text{max_abs} \times 2^S) \leq 127$; quantizes weights and bias with that same S
export/export_hex.py	Python	Writes quantized coefficients/bias/shift as <code>.mem/.hex</code> files for RTL <code>\$readmemh</code> , plus a <code>channel_config.json</code> consumed later by the PS driver (M10)

STANDALONE VERIFICATION

Reload the quantized (int8 + bias + shift) weights in a plain NumPy simulation, run them over a batch of grayscale CIFAR images doing exactly the arithmetic the hardware will do (integer multiply, sum, round-half-up rescale, saturate), and visually confirm the 4 output maps look like recognizable edge/blob-style responses rather than noise — a qualitative sanity check before this feeds the bit-exact golden model in M3. **Gate: still 100% software; no RTL or board involved yet.**

3. Golden Reference Model & Test-Vector Generation

SOFTWARE

Depends on: M2 (for CIFAR vectors); independent for hardcoded-kernel vectors

Objective: Build the single, bit-exact source of truth every RTL testbench will be checked against — implementing the locked fixed-point pipeline exactly: shared zero-padded window → per-channel multiply+bias+adder-tree → round-half-up rescale → saturate → ReLU.

File	Type	Purpose
golden_model/fixed_point_model.py	Python	Bit-exact NumPy implementation of the full datapath, parameterized by N, K, and a loaded channel_config (coefficients/bias/shift/ReLU-enable)
golden_model/test_vectors.py	Python	Generates test suites: random windows, directed corner/ boundary cases, a deliberate saturation-triggering case, the CIFAR-extracted filter set (M2), and the hardcoded Sobel/ blur filter set — writes inputs + expected outputs as .hex
golden_model/verify_by_hand.ipynb	Notebook	2–3 pixels computed by hand/calculator, cross-checked against the model's own output

STANDALONE VERIFICATION

At least 3 hand-computed pixels match the model exactly. Test-vector generation is scripted and reproducible (fixed seed → identical files, committed to the repo). **Gate: do not write a single line of RTL until this model can emit a complete expected-output file for both the hardcoded-kernel set and the CIFAR-extracted set.**

4. RTL — Configuration Register Bank & AXI4-Lite Control

HARDWARE

Depends on: M0

Objective: The per-channel storage for coefficients, bias, shift, and ReLU-enable, addressable over AXI4-Lite — built and verified before anything reads from it.

File	Type	Purpose
rtl/coeff_bias_shift_regfile.vhd	VHDL	Per-channel: 9×8-bit coefficients, signed bias register, shift register, ReLU-enable bit — ×K, addressable by channel_index×stride+tap_index
rtl/axi_lite_ctrl.vhd	VHDL	AXI4-Lite slave wrapping the register file above
tb/tb_ccoeff_bias_shift_regfile.vhd	VHDL TB	Self-checking testbench: writes then reads every register
tb/tb_axi_lite_ctrl.vhd	VHDL TB	Drives simulated AXI4-Lite write/read transactions against every channel/address

STANDALONE VERIFICATION

Every register, every channel, every tap index: write then read back, assert exact match — 100% pass, self-checking assertions, not waveform-by-eye. Confirm address decoding never aliases between channels. **Gate: this module is not touched again until it independently passes both testbenches.**

5. RTL — Shared Window Generator

HARDWARE

Depends on: M3 (test vectors), M0

Objective: The one-per-design (not one-per-channel) sliding-window mechanism every channel reads from — SRL-based, zero-BRAM target, with row/col position tracking for downstream framing signals.

File	Type	Purpose
rtl/line_buffer_srl.vhd	VHDL	SRL-based (N-1) row-delay line buffers
rtl/window_generator.vhd	VHDL	Shift-register window array + row/col position counters + AXI4-Stream tlast/frame-boundary signal generation
tb/tb_window_generator.vhd	VHDL TB	Streams a known padded test image (from M3) in, captures every window position, compares byte-exact against expected

STANDALONE VERIFICATION

Every window position across a full test image (including the zero-padded borders) matches expected byte-exact. **Gate:** run out-of-context synthesis on this module alone and confirm the utilization report shows **0 BRAM** — i.e. Vivado actually inferred SRLs rather than falling back to a flip-flop chain. Do not proceed until this is confirmed in a real synthesis report, not assumed.

6. RTL — Per-Channel Compute Datapath

HARDWARE

Depends on: M3, M4

Objective: The direct-form parallel MAC array, adder tree (with bias as a 10th term), rescale/round/saturate, and ReLU — one channel, fully verified in isolation, before any duplication.

File	Type	Purpose
rtl/mac_array.vhd	VHDL	N^2 parallel multipliers (window tap $\times$ coefficient); use_dsp attribute exposed as a generic for the M8 empirical DSP-vs-LUT test
rtl/adder_tree.vhd	VHDL	Balanced adder tree summing N^2 products + bias (10 terms for $N=3$)
rtl/rescale_round_saturate.vhd	VHDL	Programmable right-shift, round-half-up, saturate to signed output range
rtl/relu.vhd	VHDL	ReLU mux, gated by the per-channel enable bit
rtl/conv_channel.vhd	VHDL	Top-level single-channel wrapper wiring the four modules above
tb/tb_conv_channel.vhd	VHDL TB	Feeds M3's golden-model vectors directly in; bit-exact scoreboard comparison

STANDALONE VERIFICATION

100% bit-exact match against M3's golden model across the full regression suite (random, corner, saturation-triggering, CIFAR-filter, and hardcoded-kernel vectors) — zero mismatches. **Gate:** also run out-of-context synthesis on this module twice — once with DSP48 inference, once forced to LUT-fabric — and record the first real LUT/DSP numbers for the §6 empirical decision. Do not duplicate this module into an array until it independently passes 100% of the regression suite.

7. RTL — Top-Level K-Channel Integration

HARDWARE

Depends on: M5, M6

Objective: Wire one shared window generator to K instances of the verified channel datapath, pack their outputs into a single AXI4-Stream word, and validate the complete accelerator core end-to-end in simulation.

File	Type	Purpose
rtl/conv_top.vhd	VHDL	window_generator + K× conv_channel (generate loop) + axi_lite_ctrl + output-packing logic + AXI4-Stream master port + control FSM
tb/tb_conv_top.vhd	VHDL TB	Full-chip simulation: streams a complete padded test image in, unpacks the K-channel AXI4-Stream output, scoreboards against M3 for the whole image

STANDALONE VERIFICATION

100% bit-exact match across the entire regression suite, for a **complete image** (not just single windows) — both the CIFAR-extracted filter set and the hardcoded-kernel set, since both must run correctly through the same unmodified hardware. Capture waveform screenshots here (a required report deliverable). **Gate: do not proceed to Vivado/synthesis until this regression is 100% green in behavioral simulation.**

8. Vivado Block Design, AXI DMA, Synthesis & Implementation

SYSTEM

Depends on: M7

Objective: Package the verified core as reusable IP, wire it to the Zynq PS via AXI DMA in a scripted (reproducible) Vivado project, and carry it through synthesis and place-and-route to a signed-off bitstream.

File	Type	Purpose
vivado/create_project.tcl	TCL	Scripted, reproducible Vivado project creation (not a GUI-only .xpr)
vivado/package_conv_top_ip.tcl	TCL	Packages conv_top as an IP-Integrator-ready component
vivado/block_design.tcl	TCL	Zynq PS + conv_top + axi_dma + AXI interconnect, scripted block design
constraints/zedboard.xdc	XDC	Pin and clock constraints for the ZedBoard

STANDALONE VERIFICATION

Behavioral simulation of the full block design (Zynq PS simulation model + DMA + conv_top) with a test bench mimicking real PS-side AXI/DMA transactions, still scored against the M3 golden model. Then: full synthesis with the empirically-better multiplier setting from M6, review the synthesis log for latches/unexpected warnings, run implementation, confirm post-route timing ($WNS \geq 0$ or a documented lower Fmax), pull the final power report, and generate the bitstream. **Gate: re-run the full golden-model regression as a post-implementation timing simulation before touching real hardware.**

9. PetaLinux BSP & Device Tree

SYSTEM

Depends on: M8 (needs the exported hardware platform / XSA)

Objective: Build a bootable embedded-Linux image for the ZedBoard PS that exposes the custom AXI4-Lite and DMA peripherals to user-space, so Python can talk to them natively on-target.

File	Type	Purpose
petalinux/build.sh	Shell	Scripted steps: Vivado XSA export → PetaLinux project creation → config → build
petalinux/project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi	Device Tree	Overlay describing the custom conv_top AXI peripheral and DMA channel to the kernel

STANDALONE VERIFICATION

Boot the built image on the real ZedBoard, confirm Linux reaches a shell. Confirm `/sys/class/uio/` (or equivalent) entries exist for the custom peripheral. Confirm a raw `devmem` read/write against a known AXI4-Lite register succeeds. **Gate: this is the first real-hardware smoke test, deliberately kept separate from the full application (M10) so a failure here is unambiguously a BSP/device-tree issue, not an application bug.**

10. PS-Side Python Driver & On-Target Hardware Verification

SOFTWARE

Depends on: M2, M3, M9

Objective: The real, native-Python application that loads configuration, drives inference over DMA, and proves the physical board reproduces the golden model exactly.

File	Type	Purpose
ps_app/axi_driver.py	Python	Low-level AXI4-Lite register read/write via mmap/UIO
ps_app/dma_driver.py	Python	DMA descriptor setup/trigger wrapper for the AXI DMA channel
ps_app/load_config.py	Python	Loads M2's channel_config.json, writes coefficients/bias/shift/enable via axi_driver
ps_app/run_inference.py	Python	DMAs a test image in, DMAs the K-channel result back, unpacks it
ps_app/compare_golden.py	Python	Bit-exact comparison of on-target output against M3's expected-output files

STANDALONE VERIFICATION

Run the entire M3 regression suite — hardcoded kernels and CIFAR-extracted filters alike — on the physical ZedBoard, and confirm 100% bit-exact match against the golden model, on real silicon, not simulation. Log real measured latency and throughput here for the report's Table 1. **Gate: any mismatch gets replayed through M7's simulation first, to isolate a hardware-specific issue (I/O timing, CDC, power-on state) from a latent RTL bug simulation missed — never patch blind on the board.**

11. Full System Demo, Bonus Features & Report Packaging

FINAL

Depends on: M10

Objective: Assemble every required and bonus deliverable using the final, frozen numbers, and package the submission.

File	Type	Purpose
demo/edge_detection_demo.py	Python	Runs both the CIFAR-extracted filters and the hardcoded Sobel/blur set live on the board against a real test image; saves output visualizations for the report and live demo
report/	Docs/assets	Architecture + block diagram, datapath, FSM, memory organization, bit-width analysis, testbench & golden-model description, waveform screenshots, synthesis/timing/power reports, Table 1, FOM calculation, design-tradeoff discussion

STANDALONE VERIFICATION

Final regression re-run one more time end-to-end as a pre-submission sanity check. Every mandatory spec item and every bonus item (board demo, multi-kernel, ReLU, edge-detection demo) cross-checked against the original competition requirements one line at a time. Submission package dry-run: confirm every file referenced in the report actually exists in the package before hitting submit.

Appendix A. Full Project File Tree

```
cnn_accelerator/  
├─ training/                                # M1  
│   ├── dataset.py  
│   ├── model.py  
│   ├── train.py  
│   └─ evaluate.py  
├─ export/                                  # M2  
│   ├── select_filters.py  
│   ├── quantize.py  
│   └─ export_hex.py  
├─ golden_model/                            # M3  
│   ├── fixed_point_model.py  
│   ├── test_vectors.py  
│   └─ verify_by_hand.ipynb  
├─ rtl/                                     # M4-M7  
│   ├── coeff_bias_shift_regfile.vhd  
│   ├── axi_lite_ctrl.vhd  
│   ├── line_buffer_srl.vhd  
│   ├── window_generator.vhd  
│   ├── mac_array.vhd  
│   ├── adder_tree.vhd  
│   ├── rescale_round_saturate.vhd  
│   ├── relu.vhd  
│   ├── conv_channel.vhd  
│   └─ conv_top.vhd  
├─ tb/                                     # M4-M7  
│   ├── tb_coeff_bias_shift_regfile.vhd  
│   ├── tb_axi_lite_ctrl.vhd  
│   ├── tb_window_generator.vhd  
│   ├── tb_conv_channel.vhd  
│   └─ tb_conv_top.vhd  
├─ vivado/                                 # M8  
│   ├── create_project.tcl  
│   ├── package_conv_top_ip.tcl  
│   ├── block_design.tcl  
│   └─ constraints/zedboard.xdc  
├─ petalinux/                              # M9  
│   ├── build.sh  
│   └─ project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi  
├─ ps_app/                                 # M10  
│   ├── axi_driver.py  
│   ├── dma_driver.py  
│   ├── load_config.py  
│   ├── run_inference.py  
│   └─ compare_golden.py  
├─ demo/                                   # M11  
│   └─ edge_detection_demo.py
```

```
|— report/                # M11
|— requirements.txt       # M0
└— README.md             # M0
```

Master plan consolidating every architecture decision locked in the accompanying spec document. Files listed are the complete set required from first script to final bitstream and report.